TECH CHALLENGE — FASE 2 · PÓS-GRADUAÇÃO EM IA

Otimização de Rotas para Distribuição de Medicamentos e Insumos

Relatório Técnico Consolidado

Algoritmo Genético · Restrições · Integração com LLM · Comparativo · Visualizações

Projeto: RotaVital — rede de distribuição de um hemocentro na cidade de São Paulo (dados sintéticos)
Repositório do projeto: github.com/thalesgomest/fiap-ia-devs-tech-challenge-fase2-rotavital

EQUIPE

Nome	RM
Daniel Henrique Soter de Souza	rm376376
Otavio Braga Goulart	rm376980
Thales Gomes Targino	rm376365

Sumário

1. Introdução
2. Implementação do Algoritmo Genético para Roteamento
3. Estratégias para as Restrições Adicionais
4. Integração com LLMs para Instruções e Relatórios
5. Comparativo de Desempenho com Outras Abordagens
6. Visualizações e Análise das Rotas Otimizadas
7. Arquitetura da Solução
8. Limitações Gerais e Trabalhos Futuros

1. Introdução

A distribuição diária de hemocomponentes, vacinas, reagentes, medicamentos de atendimento domiciliar e insumos hospitalares é um problema logístico com consequências clínicas diretas: uma bolsa de sangue que chega tarde a um hospital ou uma vacina transportada fora da refrigeração representam risco ao paciente, não apenas custo. Ao mesmo tempo, a frota disponível é limitada e heterogênea, e cada veículo tem capacidade, autonomia e jornada próprias.

Este projeto, o **RotaVital**, modela essa operação a partir de um hemocentro localizado na região central de São Paulo, que abastece hospitais, Unidades Básicas de Saúde (UBS), laboratórios, AMAs e pacientes em casa. A partir da formulação clássica do problema do caixeiro-viajante (TSP), o problema foi estendido para um **problema de roteamento de veículos com frota heterogênea e janelas de tempo** (HFVRPTW), resolvido por um **algoritmo genético** (GA). Uma camada de **LLM** transforma as rotas otimizadas em instruções para motoristas, relatórios de eficiência e respostas a perguntas em linguagem natural.

Objetivo do projeto: desenvolver e avaliar criticamente um sistema de otimização de rotas para o contexto hospitalar, cobrindo modelagem das restrições, implementação e ajuste do GA, comparação com abordagens clássicas de roteamento e integração com LLMs. Este relatório consolida as decisões e os resultados; o detalhamento completo está na documentação do repositório (`docs/`) e no código (`src/rotavital/`).

Cenários avaliados

Os cenários são sintéticos e reprodutíveis: um script com semente fixa sorteia distritos reais de São Paulo (coordenadas aproximadas) e atribui a cada ponto um perfil de entrega (tipo de unidade, carga, peso, prioridade, janela e necessidade de refrigeração). Os nomes das unidades e as demandas são fictícios.

Cenário	Entregas	Composição	Frota (van refrig. / utilitário / moto)
sp-hemorrede-40 (principal)	**40**	**9 hospitais, 11 UBS, 5 laboratórios, 9 domiciliares, 6 AMAs**	**3 / 2 / 4**
sp-homecare-18	18	3 hospitais, 3 UBS, 10 domiciliares, 2 AMAs	1 / 1 / 2
sp-mini-8	8	2 hospitais, 2 UBS, 3 domiciliares, 1 AMA	2 / 2 / 3

Tabela 1. Cenários utilizados nos experimentos.

Veículo	Carga	Autonomia	Velocidade	Refrigerado	Custo fixo	Custo/km	Jornada
Van refrigerada	350 kg	220 km	26 km/h	Sim	R$ 180	R$ 1,90	10 h
Utilitário (carga seca)	700 kg	300 km	24 km/h	Não	R$ 140	R$ 1,50	10 h
Moto com baú térmico	30 kg	140 km	34 km/h	Sim	R$ 70	R$ 0,45	8 h

Tabela 2. Tipos de veículo. Custos e velocidades são estimativas para operação urbana, configuráveis por cenário.

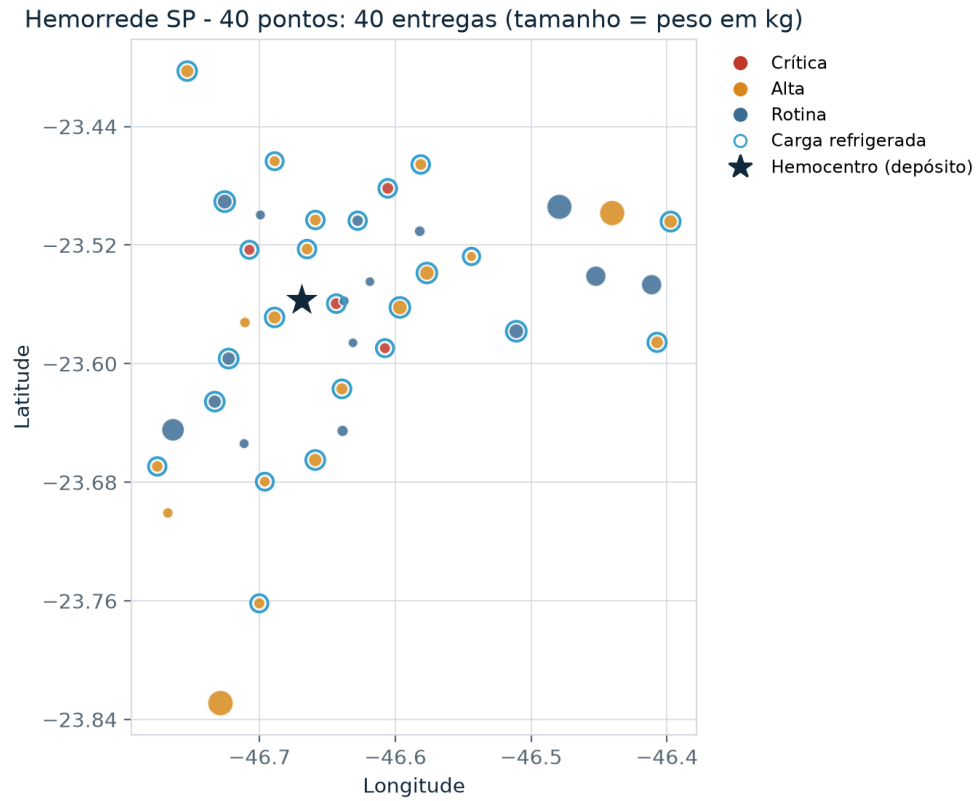


Figura 1. Entregas do cenário principal por prioridade; o anel azul indica carga refrigerada e o tamanho do ponto, o peso.

2. Implementação do Algoritmo Genético para Roteamento

2.1 Do TSP do código base ao VRP hospitalar

O código base resolve o TSP de um único veículo: o cromossomo é uma permutação das cidades e o fitness é a distância total do percurso. O RotaVital **mantém a representação por permutação**, porque ela garante que qualquer indivíduo é válido e permite usar operadores clássicos de permutação, e acrescenta quatro elementos para tratar o problema real:

1. **Decodificador Split** (Prins, 2004), que transforma a permutação em várias rotas e escolhe o veículo de cada uma;
2. **Função de custo em reais**, no lugar da distância pura, para incorporar prioridades, frota e veículos;
3. **Tratamento de frota limitada** por reparo e preços-sombra;
4. **Busca local** (2-opt e realocação), tornando o GA memético.

2.2 Representação e decodificação (Split)

O cromossomo é o "tour gigante": a ordem das 40 entregas, sem depósito e sem separadores de rota. A decisão de onde cortar a sequência e qual veículo usar fica com o Split, uma programação dinâmica que encontra, para uma ordem fixa, a divisão de menor custo:

```
V[0] = 0
V[j] = min sobre i < j e tipo k:  V[i] + custo_k(entregas i+1 … j)
```

`custo_k` é infinito quando a rota viola capacidade, autonomia, jornada ou cadeia fria para o tipo `k`. A simulação é incremental (estender a rota em uma parada custa tempo constante) e a varredura é interrompida assim que uma restrição física estoura, o que mantém a avaliação rápida: cerca de 14 mil avaliações de fitness em 24 segundos no cenário de 40 entregas. Foram consideradas e descartadas duas alternativas: separadores de rota no cromossomo (geram indivíduos inválidos após cruzamento e mutação) e cromossomo duplo ordem + veículo (dobra o espaço de busca, com a maioria das combinações inviável por carga ou refrigeração).

2.3 Função de fitness: custo do dia em reais

```
custo = Σ rotas    [ custo fixo do veículo + custo por km × distância ]
      + Σ entregas [ multa por minuto da prioridade × minutos de atraso ]
      + R$ 2.500 × veículos acima da frota disponível
```

Converter quilômetros, minutos de atraso e veículos para uma unidade comum evita uma soma ponderada com pesos arbitrários, produz um número interpretável pelo gestor ("o plano de hoje custa R$ 1.607") e permite comparar o GA e as heurísticas com exatamente a mesma régua. A multa por minuto funciona como taxa de câmbio entre pontualidade e quilometragem: R$ 40/min para entregas críticas equivale a cerca de 21 km de van, ou seja, o GA aceita um desvio dessa ordem para evitar um minuto de atraso num hemocomponente urgente.

2.4 Operadores genéticos

Componente	Escolha	Justificativa
População inicial	120 indivíduos: 10% sementes heurísticas, 90% aleatórios	Sementes (vizinho mais próximo, regra por prioridade, Clarke-Wright e variações mutadas) aceleram o início; os aleatórios preservam diversidade.
Seleção	Torneio, k = 3	Pressão seletiva moderada e independente da escala do custo.
Cruzamento	OX (Order Crossover), taxa 0,9; PMX e ERX disponíveis	OX preserva blocos contíguos, que viram trechos de rota após o Split.
Mutação	Troca, inserção ou inversão; taxa 0,25 adaptativa até 0,8	A taxa cresce 1,6× a cada 40 gerações sem melhoria e volta ao normal quando o custo melhora.
Diversidade	Elitismo 2 e 15% de imigrantes na estagnação	O melhor indivíduo nunca se perde; imigrantes renovam a população quando ela estagna.
Busca local	2-opt no melhor indivíduo a cada 10 gerações; 2-opt + realocação no final	Ajuste fino avaliado pelo custo completo (respeita janelas e prazos).
Parada	120 gerações sem melhoria, 400 gerações ou 180 s	Estagnação encerra a maioria das execuções; os demais limites garantem resposta em tempo de uso.

Tabela 3. Componentes do GA (configuração base E1). Todos os sorteios usam um gerador com semente, tornando cada execução reprodutível.

2.5 Experimentos de configuração

Para cumprir e ampliar a exigência de ao menos três experimentos, foram testadas **oito configurações** no cenário principal, cada uma mudando **um único fator** em relação à base E1 e executada com **10 sementes** diferentes (80 execuções do GA). A significância das diferenças em relação ao E1 foi verificada com os testes de Mann-Whitney e t de Welch.

Exp.	Mudança em relação ao E1	Custo médio	Desvio	Melhor	Pior	Tempo	p (M-W)
E1	**Base: pop 120, OX, mutação 0,25**	**1.630,39**	**27,93**	**1.584,48**	**1.675,86**	**24,7 s**	**—**
E2	População 40	1.665,21	53,06	1.584,24	1.767,64	6,6 s	0,10
E3	População 250	1.624,92	52,26	1.527,53	1.686,53	52,0 s	1,00
E4	Mutação fixa 0,6	1.645,31	59,88	1.572,96	1.741,27	67,1 s	0,62
E5	Cruzamento PMX	1.652,88	80,08	1.547,72	1.828,76	22,4 s	0,68
E6	Cruzamento ERX	1.614,39	80,66	1.468,03	1.731,85	53,5 s	0,85
E7	Sem busca local periódica	1.626,71	69,00	1.530,92	1.793,87	23,6 s	0,52
E8	Sem sementes heurísticas	1.627,11	61,84	1.522,91	1.753,98	23,4 s	0,73

Tabela 4. Resultados por configuração (10 sementes cada, cenário sp-hemorrede-40, custos em R$). p = valor-p do teste de Mann-Whitney contra o E1.

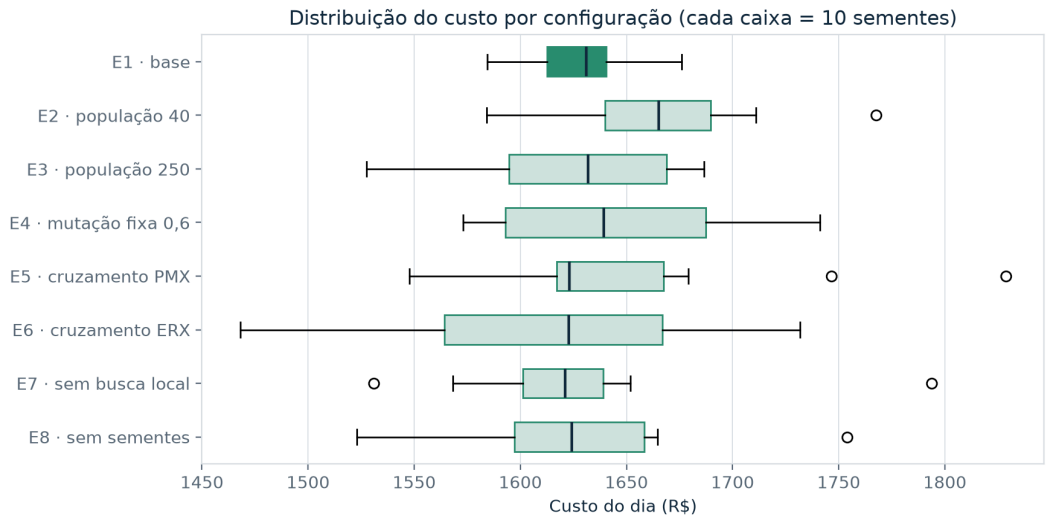


Figura 2. Distribuição do custo final das 10 execuções de cada configuração.

Nenhuma configuração difere do E1 com significância estatística (todos os valores-p acima de 0,05). As médias ficam a menos de 3% umas das outras, o que indica que o desempenho vem principalmente da modelagem (representação, Split e função de custo) e não do ajuste fino de parâmetros. A escolha do E1 como padrão se apoia em **previsibilidade**: ele tem o menor desvio padrão (R$ 27,93, metade do segundo menor) e o melhor pior caso (R$ 1.675,86). Para uso diário, uma execução sempre próxima da média vale mais do que uma configuração que às vezes acha um resultado excelente e às vezes um ruim.

- **População 40 (E2):** 3,7× mais rápida e 2,1% mais cara em média ($p \approx 0,10$, limiar da significância). É a base do modo "Rápido" da interface.
- **ERX (E6):** tem a menor média e o melhor resultado individual (R$ 1.468), mas o maior desvio e 2,2× o tempo. Justifica o uso no modo "Qualidade", em que se pode rodar mais de uma vez e ficar com o melhor.
- **PMX (E5)** tem o pior caso mais alto (R$ 1.829), coerente com preservar posições absolutas que o Split depois desfaz.

- **Busca local periódica e sementes (E7, E8)** não alteram a média de forma mensurável, mas sua retirada aumenta a dispersão. O polimento final (2-opt + realocação) roda em todas as configurações e absorve parte do efeito.

> **Sinal a observar:** a média de gerações executadas ficou entre 308 e 400; várias execuções atingiram o teto de 400 gerações ainda melhorando lentamente. Isso indica que um teto maior traria ganhos pequenos adicionais, ao custo de mais tempo — um compromisso que hoje favorece o tempo de resposta da interface.

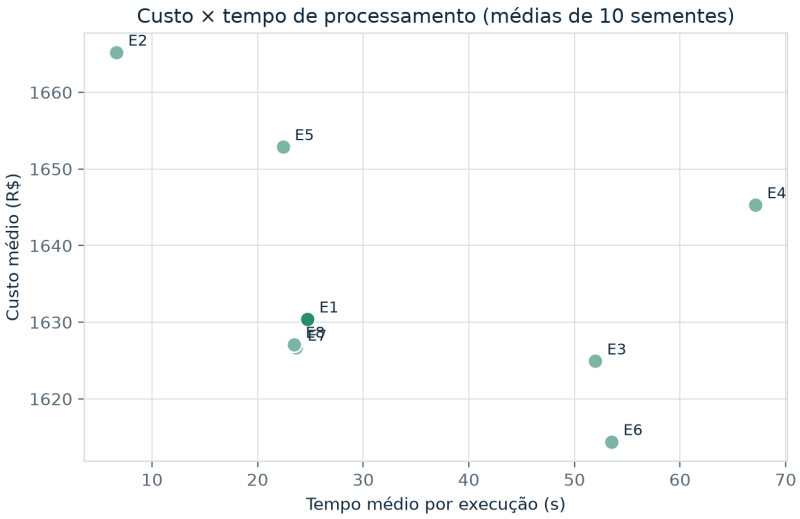


Figura 3. Custo médio versus tempo médio de processamento por configuração.

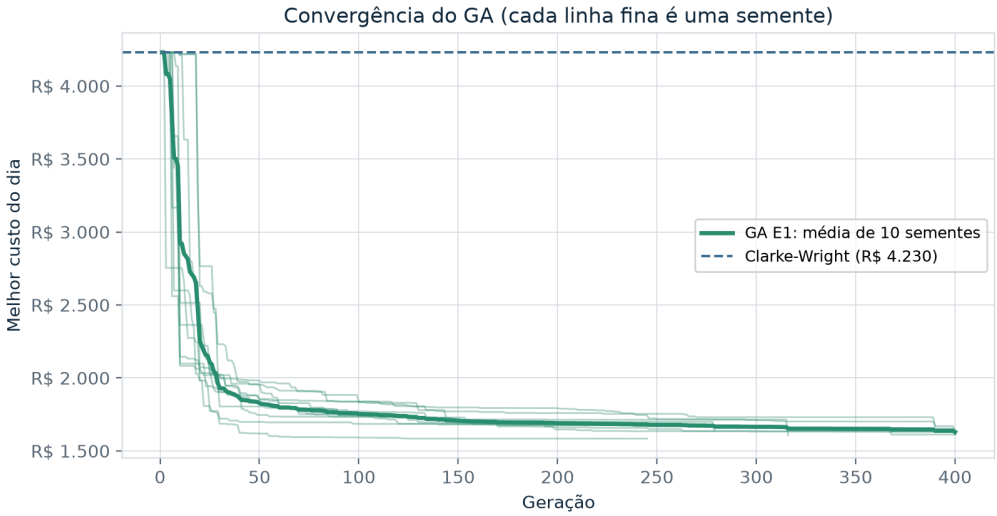


Figura 4. Convergência do E1: a população parte do custo do Clarke-Wright (incluído como semente) e cai abaixo da metade nas primeiras 50 gerações.

3. Estratégias para as Restrições Adicionais

Cada restrição foi classificada como **rígida** (fisicamente impossível de violar: a rota é descartada pelo Split para aquele tipo de veículo) ou **flexível** (indesejada, mas possível: vira multa em reais). As flexíveis criam um gradiente para a busca — uma solução com 5 minutos de atraso é melhor que uma com 50 — enquanto as rígidas garantem que nenhum plano entregue seja fisicamente inexecutável.

Restrição	Tipo	Estratégia de implementação
Prioridades (crítica, alta, rotina)	Flexível	Prazo efetivo = menor entre o fim da janela e o SLA da prioridade (crítica 2 h, alta 4 h após 07:00). Atraso medido no início do atendimento, com multa de R$ 40, R$ 10 ou R$ 1,50 por minuto. Visitas agendadas para depois do SLA seguem a janela combinada.
Capacidade de carga	Rígida	Carga acumulada verificada a cada parada no Split; ao exceder a capacidade do tipo, a varredura para aquele tipo é interrompida.
Autonomia	Rígida	Distância acumulada mais o retorno ao depósito comparada à autonomia do veículo. Distância = Haversine × 1,35 (fator de sinuosidade urbana).
Múltiplos veículos (VRP)	Rígida + flexível	O Split decide quantas rotas e qual dos três tipos de veículo faz cada uma. A quantidade disponível por tipo é tratada pelo mecanismo de frota limitada abaixo.
Janelas de horário	Flexível	Chegada antes da abertura gera espera (sem multa); chegada após o prazo gera atraso (com multa).
Cadeia fria	Rígida	Parada refrigerada torna a rota inviável para veículos não refrigerados (utilitário).
Jornada do motorista	Rígida	Retorno ao depósito após o fim da jornada do tipo (10 h vans e utilitários, 8 h motos) torna a rota inviável.

Tabela 5. Restrições e estratégias. Todas as grandezas de tempo são tratadas em minutos, para evitar comparações entre unidades diferentes.

Validação do cenário

Antes de otimizar, o sistema rejeita cenários sem solução possível: paradas que nenhum veículo comporta (por peso ou refrigeração) ou que nenhum veículo alcança e volta dentro da autonomia e da jornada geram um erro explicativo, em vez de uma otimização que nunca converge.

Frota limitada: reparo, preços-sombra e multa

O Split clássico supõe frota ilimitada. No cenário principal, com muita carga refrigerada, isso levava a soluções com quatro vans refrigeradas quando só existem três. A estratégia adotada tem três camadas, aplicadas em cada avaliação de fitness:

1. **Reparo guloso:** rotas de um tipo excedido são transferidas para um tipo com vaga, se continuarem viáveis, escolhendo a troca de menor aumento de custo;
2. **Preços-sombra:** se ainda houver excesso, o Split é refeito com o custo fixo do tipo excedido acrescido de R$ 833 (um terço da multa), até três vezes; o Split passa a preferir outros veículos naturalmente;

3. **Multa:** o excesso que restar paga R$ 2.500 por veículo — mais de quatro vezes o custo do dia mais caro de uma van (R$ 180 + 220 km × R$ 1,90 ≈ R$ 598) —, e a interface sinaliza o veículo como excedente.

A solução final é escolhida pelo custo real, sem os preços-sombra. Essa estratégia é o que diferencia o GA das heurísticas no comparativo (Seção 5): todas as heurísticas excedem a frota, e o GA é a única abordagem que entrega um plano que cabe nos veículos existentes.

4. Integração com LLMs para Instruções e Relatórios

4.1 Funcionalidades

Requisito	Implementação
Instruções para motoristas e equipes	Folha de rota por veículo: verificações antes da saída, sequência de entregas com horário, janela, prazo e carga, alertas de prioridade crítica e cadeia fria.
Relatórios diários/semanais	Relatório da operação (eficiência, economia frente às heurísticas, riscos) e relatório consolidado de 1 ou 7 dias a partir do histórico de execuções em SQLite.
Sugestões de melhoria	Seção de recomendações nos relatórios, apoiada em regras sobre os indicadores (frota insuficiente, ocupação baixa, esperas longas, críticas fora do SLA) entregues à LLM como evidência.
Perguntas em linguagem natural	Assistente com *function calling*: a LLM consulta 8 ferramentas sobre os dados reais da execução em vez de receber todo o JSON.

Tabela 6. Funcionalidades de LLM exigidas e como foram atendidas.

4.2 Provedores e arquitetura da camada

A camada define uma interface comum (`LLMProvider`) com adaptadores intercambiáveis por variável de ambiente: **Ollama** (padrão: modelo local Qwen3 4B, gratuito e sem chave), **qualquer API compatível com OpenAI** (OpenAI, Groq, OpenRouter, Gemini), **Claude** (Anthropic, via SDK oficial) e um **modo offline** com textos determinísticos. Cada chamada tem prazo total configurável (`LLM_TIMEOUT_S`). O comportamento em caso de falha também é configurável: com a contingência ativa, o sistema usa os textos determinísticos e os identifica como offline; com ela desativada (`LLM_FALLBACK_OFFLINE=0`), a falha chega à interface como erro (HTTP 503), para que uma resposta de contingência nunca seja confundida com uma resposta do modelo. Nenhuma credencial fica no código.

4.3 Ferramentas do assistente (function calling)

Ferramenta	Exemplo de pergunta atendida
`resumo_execucao` / `listar_rotas`	"Quantos veículos saem hoje e com que ocupação?"
`detalhar_rota(veiculo)`	"Qual a sequência da MOTO #1?"
`buscar_parada(nome)`	"Quando chega a entrega do Hospital Perus?"
`entregas_em_risco(folga, prioridade)`	"Alguma entrega crítica corre risco de atraso?"
`simular_atraso(veiculo, minutos)`	"Se a VAN-REF #1 sair 30 minutos atrasada, o que acontece?"
`comparar_abordagens` / `historico(dias)`	"Quanto economizamos frente ao Clarke-Wright?" / "Como foi a semana?"

Tabela 7. Ferramentas consultadas pela LLM. Cada resposta exibe na interface quais ferramentas foram usadas e o que retornaram.

4.4 Engenharia de prompts

- **Papel e domínio:** prompt de sistema define a assistente de logística do hemocentro e o público (profissionais de saúde e logística).
- ***Grounding* explícito:** "use somente os dados fornecidos", "copie números e horários exatamente", "se não estiver nos dados, diga que não está disponível".
- **Dados estruturados e enxutos:** JSON apenas com os campos relevantes de cada tarefa.
- **Formato de saída fixo** com seções nomeadas e um **exemplo de linha** (one-shot) para a folha de rota.
- **Evidências pré-calculadas** (entregas em risco, sugestões por regra): a LLM interpreta em vez de calcular.
- **Respostas estáveis:** temperatura 0, semente fixa e raciocínio interno do modelo desativado (`think=false`), com limites de contexto (8.192 tokens) e de saída (512 tokens).
- **Regras de escopo no prompt do chat:** consultar as ferramentas da execução atual em vez de reaproveitar números de respostas anteriores, não confundir prioridade alta com crítica e reconhecer quando a informação não existe nos dados (por exemplo, trânsito em tempo real).
- **Prompts versionados** em arquivos `.md` , separados do código.

4.5 Avaliação da qualidade dos textos

Cada texto gerado recebe uma nota de 0 a 100, armazenada no histórico com o provedor e o modelo: (i) **fidelidade numérica** — todos os números e horários citados são conferidos contra os dados de origem, detectando alucinação de forma objetiva; (ii) **cobertura e ordem** — nas instruções, todas as paradas devem aparecer e na sequência planejada; (iii) **LLM como juiz** (opcional) — rubrica de 1 a 5 para clareza, correção, acionabilidade e segurança do paciente.

```
# Folha de rota - MOTO #1
Saída: 07:00 · Retorno previsto: 12:32 · Distância: 51.87 km · Carga total: 27.1 kg
## Antes de sair
- Conferir as cargas na ordem inversa de entrega.
- Verificar temperatura do baú refrigerado e registrar no checklist.
## Sequência de entregas
1. Hospital Perus - chegada 07:46 · janela 07:00-18:00 · prazo 11:00 · entregar 21.4 kg de
Hemocomponentes. ❄ manter refrigerado
2. Paciente domiciliar Freguesia do Ó - chegada 08:27 · janela 12:00-14:00 · prazo 14:00 · entregar 5.7 kg
de Medicamentos home care. aguardar abertura (213.4 min)
## Atenção
- Nenhuma entrega com risco de atraso no planejamento.
```

Figura 5. Folha de rota real da solução do GA (cenário principal), gerada em modo de contingência. Avaliação: fidelidade numérica 15/15, cobertura e ordem 100%, nota 100.

4.6 Validação com modelo real: Qwen3 4B

O modelo local adotado é o **Qwen3 4B**, na quantização Q4_K_M, importado do arquivo GGUF oficial da Qwen sem nenhum ajuste fino e servido pelo Ollama. A revisão do arquivo e o *digest* do modelo carregado ficam registrados, e a API expõe o *digest* em `/api/status`, o que permite confirmar que as respostas vieram exatamente do modelo documentado.

A validação seguiu o mesmo caminho de um usuário: uma otimização curta no cenário `sp-mini-8` (população 20, 10 gerações, semente 2026) e três perguntas enviadas pela API do chat. O verificador rejeita respostas do modo offline, de modo que só respostas efetivamente geradas pelo Qwen3 contam como evidência.

Pergunta	Ferramenta chamada	Tempo	Revisão da resposta
Quais entregas críticas estão em risco?	`entregas_em_risco`	150,1 s	**Falha semântica:** apresentou uma entrega de prioridade *alta*, com folga de 8,5 min, como se fosse crítica. Não havia entregas críticas em risco.
Distância e retorno da MOTO #1	`detalhar_rota`	65,7 s	Correta: 35,82 km e retorno às 12:28, conferidos com os dados da execução.
Como está o trânsito em tempo real?	nenhuma	15,6 s	Correta: reconheceu que a informação não existe nos dados.

Tabela 8. Primeira rodada de perguntas reais ao Qwen3 4B. Os tempos incluem o carregamento do modelo e as chamadas de ferramenta e dependem do ambiente de execução; não constituem um benchmark de desempenho.

Nas três perguntas o modelo escolheu a ferramenta adequada (ou nenhuma, quando não havia dado a consultar), e os números citados vieram das ferramentas. A falha da primeira pergunta foi de **interpretação**: a ferramenta devolveu as entregas em risco de todas as prioridades e o modelo não distinguiu alta de crítica. A correção foi feita no código, não no modelo:

- `entregas_em_risco` passou a aceitar um filtro de prioridade;
- quando a pergunta menciona uma única categoria, o assistente fixa esse filtro, e o modelo não consegue trocá-lo (uma consulta sobre críticas não pode virar uma consulta sobre altas);

- zero ocorrências passou a ser devolvido como uma contagem calculada, e não como ausência de informação;
- a regra de não confundir prioridade alta com crítica foi incluída no prompt do chat, e testes de regressão passaram a cobrir esse caso.

No reteste, feito pela interface web, o Qwen3 chamou `entregas_em_risco(folga_minima_min=15, prioridade=critica)` e respondeu corretamente que não havia entregas críticas em risco (Figura 6). A suíte automatizada passou de 46 para **60 testes**, incluindo testes de contrato do provedor Ollama: parâmetros enviados, `think=false`, respostas vazias ou inválidas tratadas como erro, *digest* do modelo e erro HTTP 503 quando a contingência está desativada.

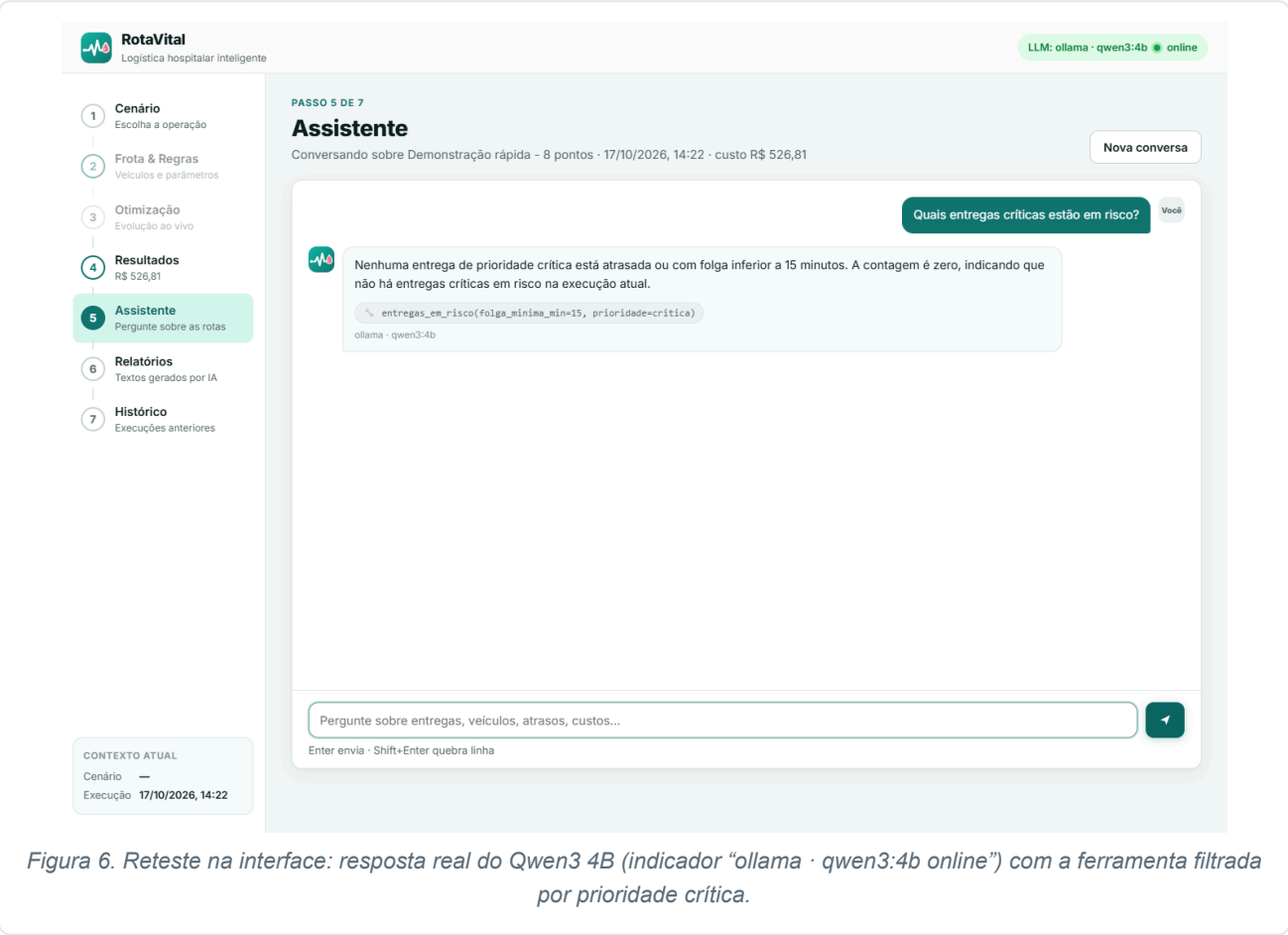


Figura 6. Reteste na interface: resposta real do Qwen3 4B (indicador "ollama · qwen3:4b online") com a ferramenta filtrada por prioridade crítica.

> **Alcance desta validação:** foram três perguntas de chat e um reteste sobre o caso que motivou a correção. O resultado mostra que a integração funciona de ponta a ponta com um modelo real e que a combinação de ferramentas com filtros no código reduz erros de interpretação, mas **não mede a acurácia geral** do modelo nem garante que respostas futuras estarão corretas. As folhas de rota e os relatórios gerados pelo Qwen3 ainda não passaram por uma avaliação sistemática com as métricas da Seção 4.5; a Figura 5 mostra a folha gerada pelo modo de contingência.

5. Comparativo de Desempenho com Outras Abordagens

Foram escolhidas quatro abordagens de referência, cada uma respondendo a uma pergunta diferente sobre o GA. Todas usam a mesma função de custo, o mesmo cenário e as mesmas restrições.

Abordagem	Funcionamento e origem	Pergunta que responde
Aleatório (melhor de 30)	30 permutações sorteadas, divididas pelo mesmo Split; fica a melhor.	O GA aprende algo ou é sorte?
Regra manual por prioridade	Ordena por prioridade, prazo e distância (regra de despacho, na linha da EDD de Jackson, 1955).	Quanto se ganha sobre a prática de um despachante?
Vizinho mais próximo + Split	Heurística construtiva gulosa clássica do TSP, com o mesmo Split do GA.	Quanto vale a busca evolutiva, com o mesmo decodificador?
Clarke-Wright (1964)	Algoritmo das economias, referência do VRP. Adaptado para só unir rotas se o custo (com atrasos) não subir.	O GA supera a referência clássica?

Tabela 9. Abordagens de comparação.

Abordagem	Operacional	Multa atraso	Multa frota	Custo total	Veículos	km	No prazo
Vizinho mais próximo	2.478,94	0,00	7.500,00	9.978,94	10	585,5	100%
Aleatório (30)	3.018,00	1.893,12	5.000,00	9.911,12	11	996,6	90,0%
Regra manual	2.781,48	604,40	5.000,00	8.385,87	11	908,3	92,5%
Clarke-Wright	1.729,69	0,00	2.500,00	4.229,69	8	446,3	100%
Algoritmo genético (E1, semente 1)	**1.607,33**	**0,00**	**0,00**	**1.607,33**	**7**	**523,1**	**100%**

Tabela 10. Composição do custo no cenário principal (R$). Com 10 sementes, o GA tem custo médio de R$ 1.630,39 ± 27,93 e 99,75% das entregas no prazo.

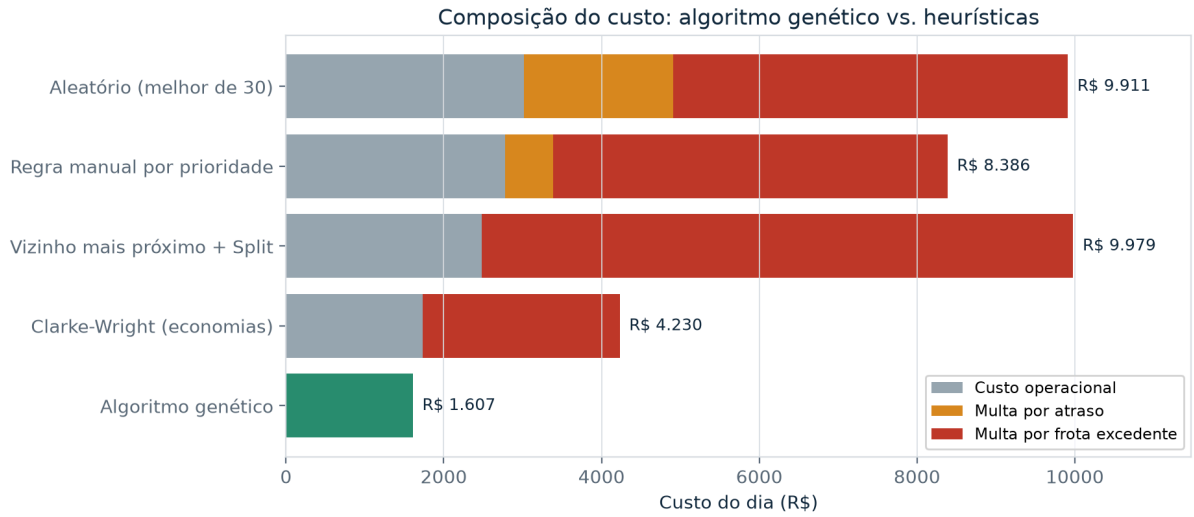


Figura 7. Composição do custo de cada abordagem.

Interpretação

- **No custo total, o GA fica 61,5% abaixo do Clarke-Wright** (média de 10 sementes). Mesmo a pior das 10 execuções (R$ 1.675,86) fica 60,4% abaixo.
- **A maior parte dessa diferença vem da frota:** todas as heurísticas montam ao menos uma rota com um veículo inexistente; o Clarke-Wright usa quatro vans refrigeradas para três disponíveis. O GA é a única abordagem que cabe na frota real.
- **Sem as multas, a vantagem é de cerca de 7%:** R$ 1.607 contra R$ 1.730 de custo operacional, com um veículo a menos (7 contra 8).
- **O GA não ganha por rodar menos:** percorre 523 km, mais que os 446 km do Clarke-Wright, e emite praticamente o mesmo CO_2 (101 contra 100 kg). O ganho está na combinação de veículos e na consolidação de cargas (ocupação média de 78%).
- **As heurísticas simples falham de formas distintas:** a regra manual atende urgências primeiro sem olhar a geografia (908 km e 3 entregas atrasadas); o vizinho mais próximo é pontual mas fragmenta a operação em 10 veículos.

Cenário	Clarke-Wright	GA (média de 10)	Desvio	Redução	No prazo (GA)
sp-mini-8	622,82	490,53	7,92	21,2%	100%
sp-homecare-18	3.537,75	806,63	59,11	77,2%	98,9%
sp-hemorrede-40	**4.229,69**	**1.630,39**	**27,93**	**61,5%**	**99,75%**

Tabela 11. Generalização para os demais cenários (configuração E1, 10 sementes, R$).

A vantagem aparece nos três cenários. No cenário pequeno, o Clarke-Wright não paga nenhuma multa, então o ganho de 21% vem só da qualidade das rotas. No cenário de home care, com frota enxuta e janelas curtas, o Clarke-Wright paga R$ 2.500 de multa de frota e R$ 115 de atraso; a capacidade do GA de encaixar a operação na frota disponível amplia a diferença para 77%.

6. Visualizações e Análise das Rotas Otimizadas

A interface web exibe as rotas em mapa interativo (Leaflet), a convergência ao vivo e uma linha do tempo por veículo. As figuras abaixo reproduzem essas visualizações para a solução do GA no cenário principal (configuração E1, semente 1), gerada pelo script `scripts/gerar_figuras.py`.

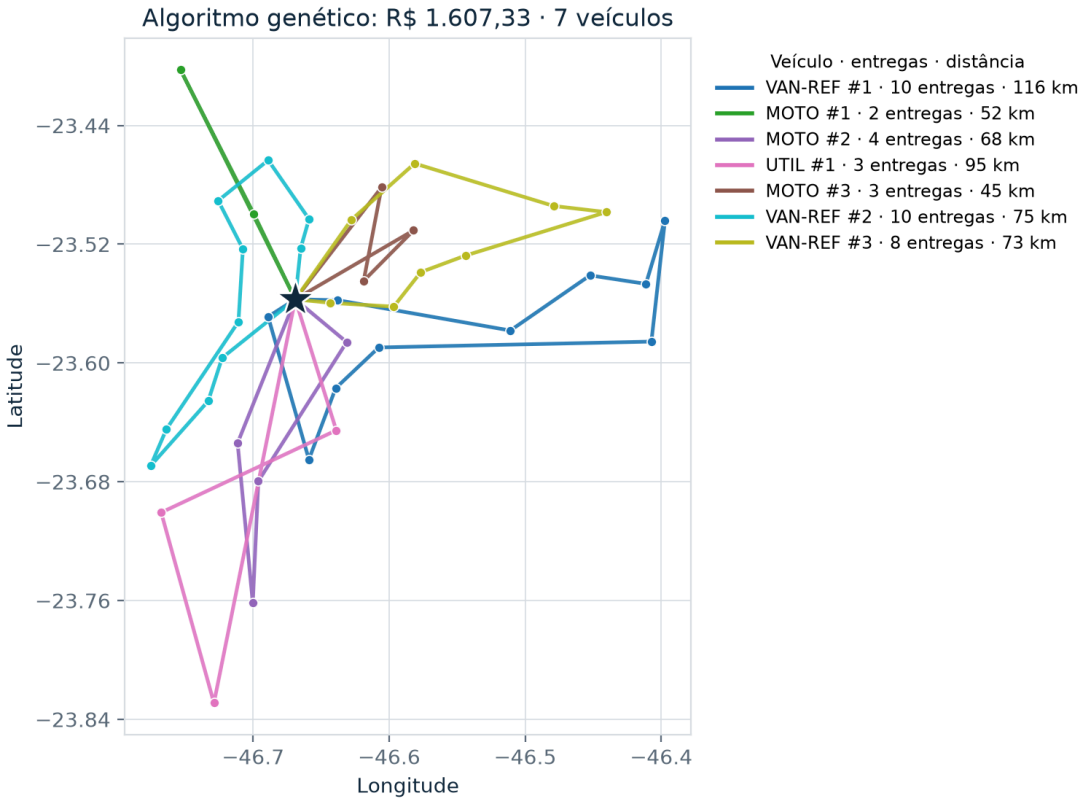


Figura 8. Rotas do algoritmo genético: 7 veículos, R$ 1.607,33.

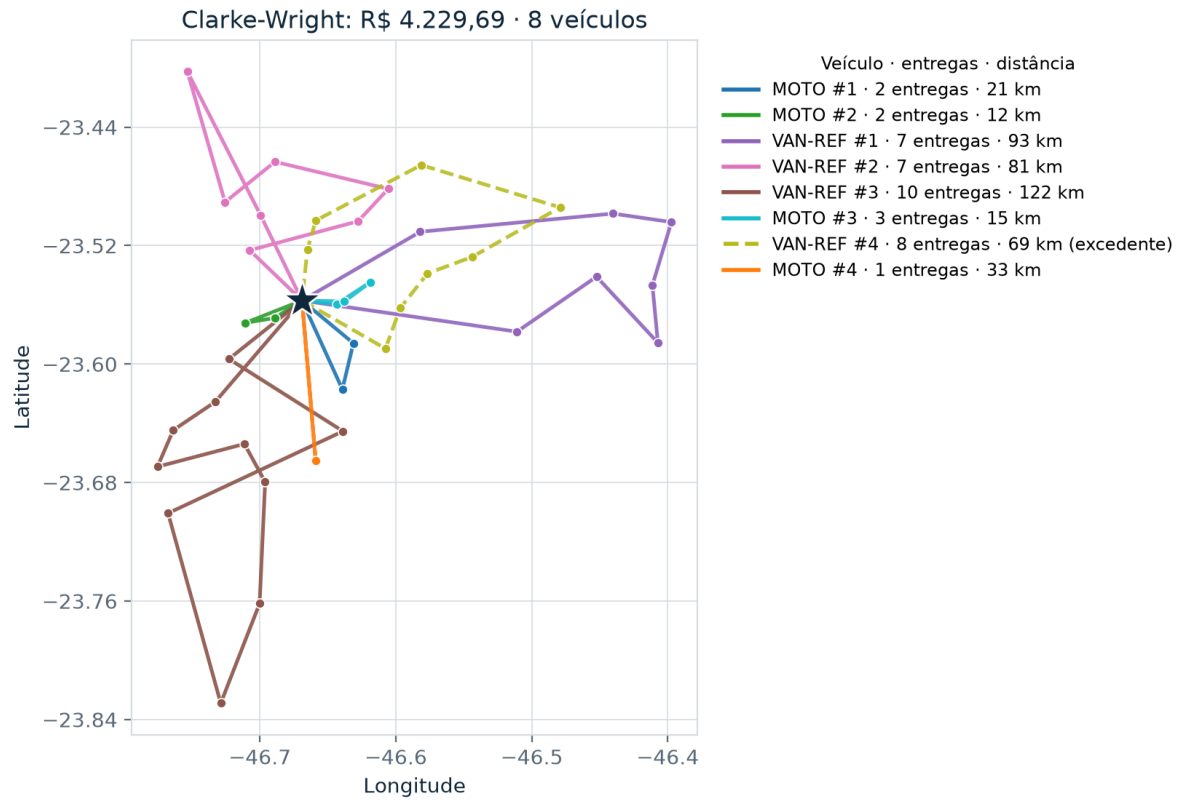


Figura 9. Rotas do Clarke-Wright: 8 veículos, incluindo uma quarta van refrigerada inexistente (linha tracejada).

Veículo	Entregas	Críticas	Refrig.	km	Saída → retorno	Ocupação	Autonomia usada
VAN-REF #1	10	1	7	116,1	07:00 → 13:33	83%	53%
VAN-REF #2	10	1	8	74,8	07:00 → 11:47	67%	34%
VAN-REF #3	8	1	6	72,7	07:00 → 11:39	97%	33%
UTIL #1	3	0	0	95,0	07:00 → 13:54	20%	32%
MOTO #1	2	0	1	51,9	07:00 → 12:32	90%	37%
MOTO #2	4	0	2	67,7	07:00 → 11:57	100%	48%
MOTO #3	3	1	1	45,0	07:00 → 13:09	90%	32%

Tabela 12. Rotas da solução do GA. Ocupação = carga / capacidade.

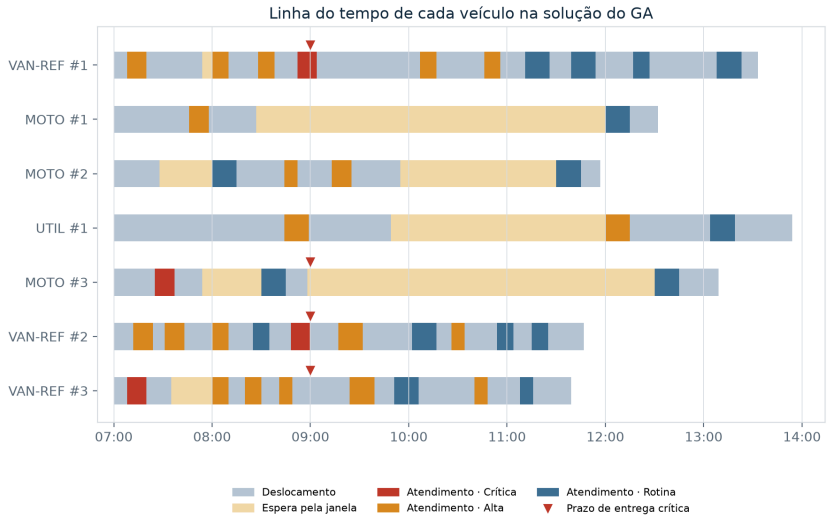


Figura 10. Linha do tempo de cada veículo: deslocamento, espera pela janela e atendimento por prioridade; triângulos marcam o prazo das entregas críticas.

Análise

- **Divisão de papéis coerente:** as três vans concentram as entregas refrigeradas (21 de 25) e três das quatro críticas; as motos levam cargas pequenas com ocupação de 90% a 100%; o único utilitário usado atende as AMAs, de carga seca e volumosa.
- **Todas as 4 entregas críticas chegam antes das 09:00** e nenhuma entrega fica atrasada; todos os veículos retornam até 13:54, bem dentro das jornadas.
- **Folga de frota:** dos 9 veículos disponíveis, 7 são usados — um utilitário e uma moto ficam livres para urgências não planejadas.
- **Esperas longas:** algumas rotas incluem esperas de horas por janelas domiciliares (a MOTO #1 aguarda 3,5 h). Como a função de custo não penaliza a espera, o GA aceita esse tempo ocioso quando ele economiza um veículo. É a principal oportunidade de melhoria do modelo (Seção 8).
- **Utilitário subutilizado** (20% de ocupação): a carga seca do dia é pequena; em dias de reposição maior de insumos, esse veículo absorveria o aumento sem mudar a estrutura das rotas.

7. Arquitetura da Solução

A implementação em nuvem (opcional no enunciado) **não foi adotada** nesta fase. A solução roda localmente, em ambiente virtual Python 3.11, com todos os comandos padronizados por um `Makefile` multiplataforma (`make demo` prepara o ambiente, roda os testes e sobe a aplicação).

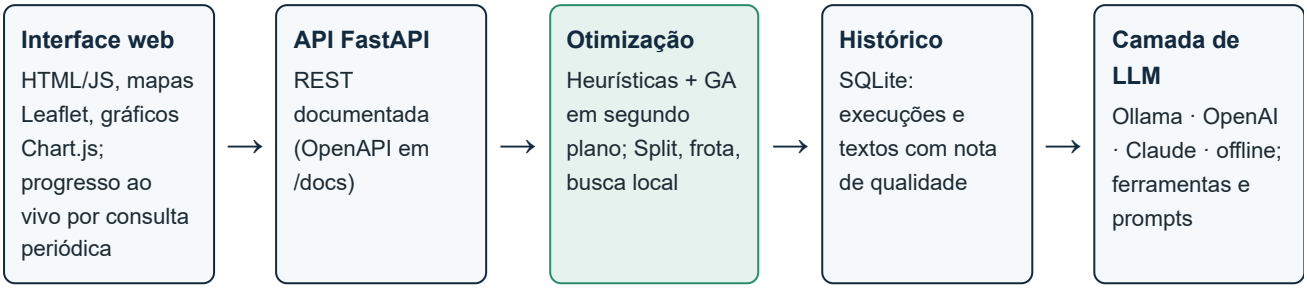


Figura 11. Componentes da solução e fluxo de uma otimização.

Decisão	Motivo
Núcleo de otimização independente	O GA não conhece a API nem a LLM; é usado por linha de comando (experimentos), testes e interface.
FastAPI + página estática	API documentada automaticamente, interface sem etapa de build, progresso incremental da otimização.
Python puro no núcleo	O Split é sequencial; listas Python e cache de fitness bastam (cerca de 25 s para 40 entregas).
Uma thread por otimização	Infraestrutura mínima; o contrato de progresso permitiria trocar por uma fila de tarefas sem mudar a API.
SQLite para histórico	Zero configuração; suficiente para relatórios por período.

Tabela 13. Principais decisões de arquitetura.

Qualidade e monitoramento

O projeto inclui 60 testes automatizados (pytest) que verificam as propriedades centrais do otimizador — visita única de cada entrega, respeito à capacidade e à cadeia fria, igualdade entre o fitness e o custo da solução detalhada, validade das permutações geradas pelos operadores e reprodutibilidade por semente —, além dos fluxos de LLM (incluindo os contratos do provedor Ollama) e da API. Cada execução registra gerações, avaliações de fitness, tempo, motivo de parada e curva de convergência; cada texto da LLM é armazenado com provedor, modelo e nota de qualidade. Um `Dockerfile` e um `docker-compose.yml` (aplicação + Ollama) foram preparados como base para uma futura implantação em contêiner, mas não foram validados em ambiente de nuvem nesta fase.

8. Limitações Gerais e Trabalhos Futuros

Limitações consolidadas

- **Dados sintéticos e custos estimados:** os valores absolutos em reais são ilustrativos; a comparação relativa entre abordagens é a conclusão robusta.
- **Distâncias aproximadas:** linha reta × 1,35, sem a malha viária real; trechos com rios, serras ou vias expressas ficam mal representados.
- **Velocidade média fixa** por tipo de veículo, sem variação de trânsito por horário.
- **Uma viagem por veículo:** o veículo não volta ao depósito para recarregar e sair de novo, o que, somado à espera não penalizada, explica as longas esperas observadas na Seção 6.
- **Frota tratada por reparo:** o Split não considera a quantidade de veículos diretamente; o reparo e os preços-sombra resolvem os casos observados, mas não garantem a divisão ótima com frota limitada.
- **Teto de gerações:** parte das execuções atingiu 400 gerações ainda melhorando.
- **Avaliação limitada da LLM:** o Qwen3 4B foi validado em três perguntas de chat e um reteste; folhas de rota e relatórios gerados pelo modelo não foram avaliados de forma sistemática, e o filtro de prioridade foi ajustado a partir de um caso já conhecido.
- **Ajuste por um fator de cada vez:** interações entre parâmetros (por exemplo, população × cruzamento) não foram exploradas.

Trabalhos futuros

- Distâncias e tempos reais por ruas (OSRM ou serviço de rotas) e tempo de viagem dependente do horário.
- Múltiplas viagens por veículo e penalização do tempo ocioso na função de custo.
- Split com controle de frota por tipo de veículo (rótulos de recursos).
- Avaliação sistemática das folhas de rota, relatórios e respostas do chat gerados pelo Qwen3, com um conjunto maior de perguntas e as métricas já implementadas, e comparação com outros modelos e provedores.
- Calibração de custos e multas com dados de uma operação real e validação com equipes de logística hospitalar.
- Implantação em nuvem a partir dos contêineres preparados, com fila de tarefas para otimizações concorrentes.

Relatório gerado a partir do código e dos experimentos deste repositório. Experimentos: `make experimentos SEMENTES=10` (cenário sp-hemorrede-40; demais cenários com `--apenas E1`). Figuras: `scripts/gerar_figuras.py`. Todos os números citados foram obtidos por execução real.